SANJIVANI UNIVERSITY

DEPARTMENT OF CYBER SECURITY

CAPSTONE PROJECT REPORT

Malware Analysis Sandbox Using VMware

Submitted by
Sarthak Chalak
PRN: 2125UCSM1029
Class: SY Cyber Security

Subject: NPC
Academic Year: 2026–2027

Abstract

This capstone activity demonstrates the use of a virtual machine as a controlled environment for observing a Python test script. Kali Linux was run in VMware Workstation with its network adapter configured for Host-only networking. A clean virtual-machine snapshot was created before the experiment. The selected Python script displayed basic system information, including the operating system, Python version, current user, and date and time. Its SHA-256 hash was checked on VirusTotal, which returned 0 detections out of 61 engines for the submitted test script at the time of checking. The original VM snapshot was then restored and the network route was checked. The activity illustrates isolation, basic observation, hash-based lookup, and snapshot-based recovery.

1. Introduction

Malware analysis involves examining suspicious files or programs to understand their behavior and potential impact. A sandbox is a controlled environment in which software can be executed and observed while limiting its access to other systems. Virtualization platforms such as VMware Workstation make it possible to create isolated virtual machines and save their state using snapshots.

For this activity, Kali Linux was used as the guest operating system. The VM was configured with Host-only networking and no default internet route. A Python system-information test script was executed, its output was observed, and its SHA-256 hash was checked on VirusTotal. The VM was subsequently restored to its clean snapshot.

2. Objectives

1. Create and configure a Kali Linux virtual machine using VMware Workstation.
2. Restrict the VM's network connectivity for the controlled experiment.
3. Create a clean snapshot before running the test script.
4. Execute a Python script that displays basic system information.
5. Observe the script's output and execution behavior.
6. Generate the script's SHA-256 hash and check for an existing VirusTotal report.
7. Restore the clean snapshot and verify the restored environment.
8. Explain the value and limitations of sandboxing and snapshot-based recovery.

3. Tools and Technologies

Tool / Technology	Purpose
VMware Workstation	Create and manage the virtual machine and snapshots.
Kali Linux	Guest operating system used for the practical activity.
Python 3	Execute the system-information test script.
platform module	Read operating-system and Python-version information.
datetime module	Display the current date and time.
os module	Read the USER environment variable.
Linux terminal commands	Inspect network configuration and system state.
SHA-256	Generate a file hash for identification.
VirusTotal	Check whether the submitted file hash has an available analysis result.

4. Experimental Environment and Network Isolation

A Kali Linux virtual machine was used in VMware Workstation. The virtual network adapter was configured as Host-only rather than NAT or Bridged. The observed interface was eth0 with an address in the 192.168.147.0/24 subnet. The route table showed the connected subnet but no default route.

The following commands were used to inspect connectivity:

```
ip addr
ip route
ping -c 4 8.8.8.8
```

The ping test returned “ping: connect: Network is unreachable”. The route table contained the connected Host-only subnet and no default gateway. This confirms that the VM had no default internet route during the test. Host-only networking is not equivalent to complete isolation: the host and other VMs on the same Host-only network may still be able to communicate with this VM.

Figure 1. VMware Host-only network configuration.

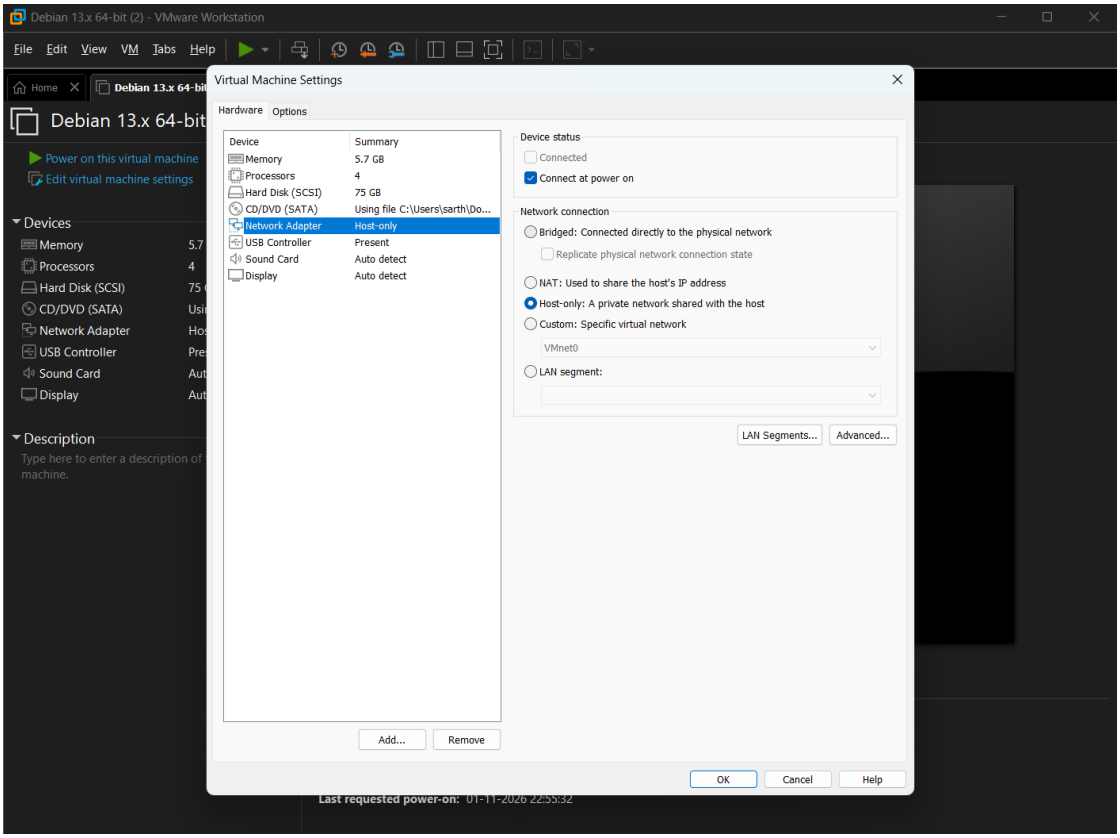
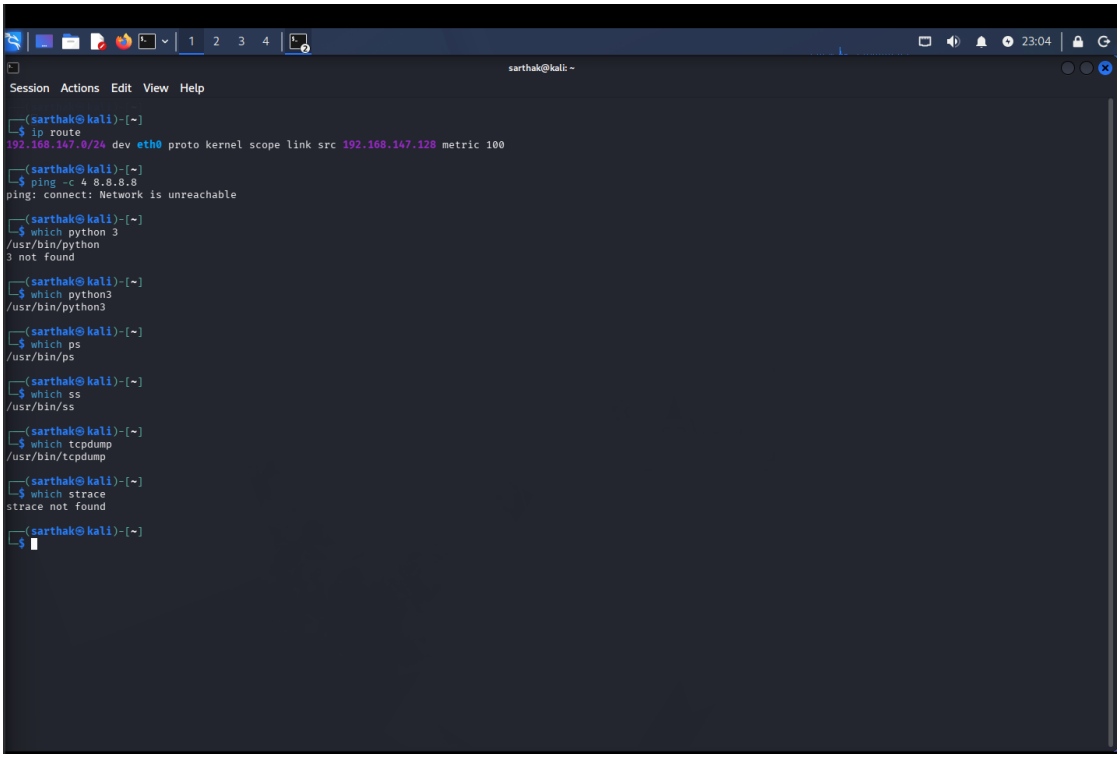


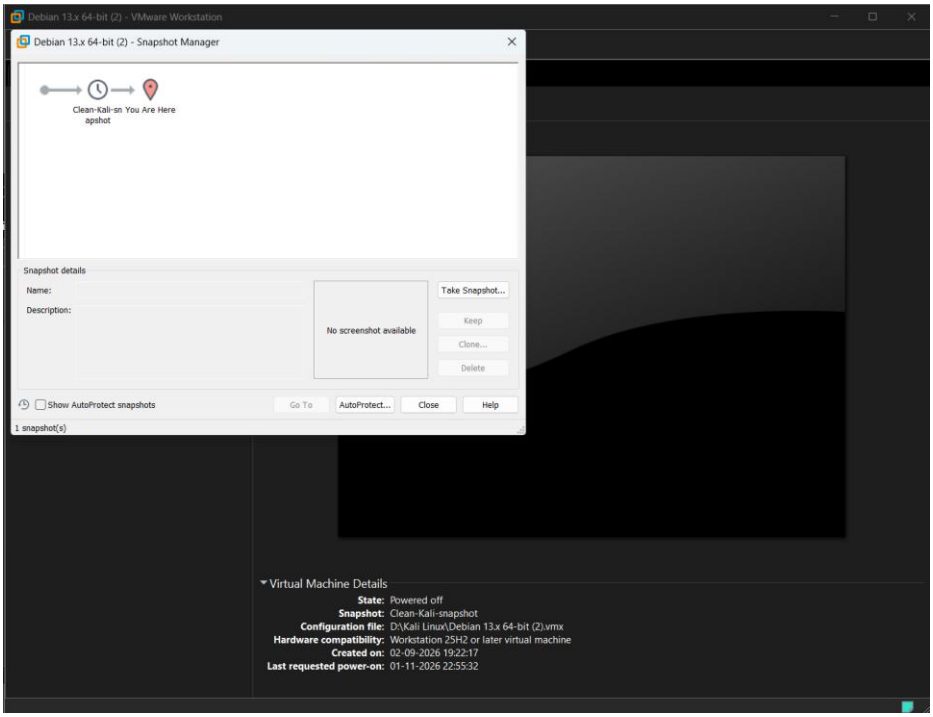
Figure 2. Kali Linux network configuration and failed internet connectivity test.



5. Clean Snapshot Creation

Before executing the test script, a snapshot of the clean Kali Linux virtual machine was created in VMware Workstation. The snapshot was named “Clean-Kali-Before-Experiment”. This saved state provided a recovery point to which the VM could be returned after the practical activity.

Figure 3. Clean-Kali-Before-Experiment snapshot in VMware.



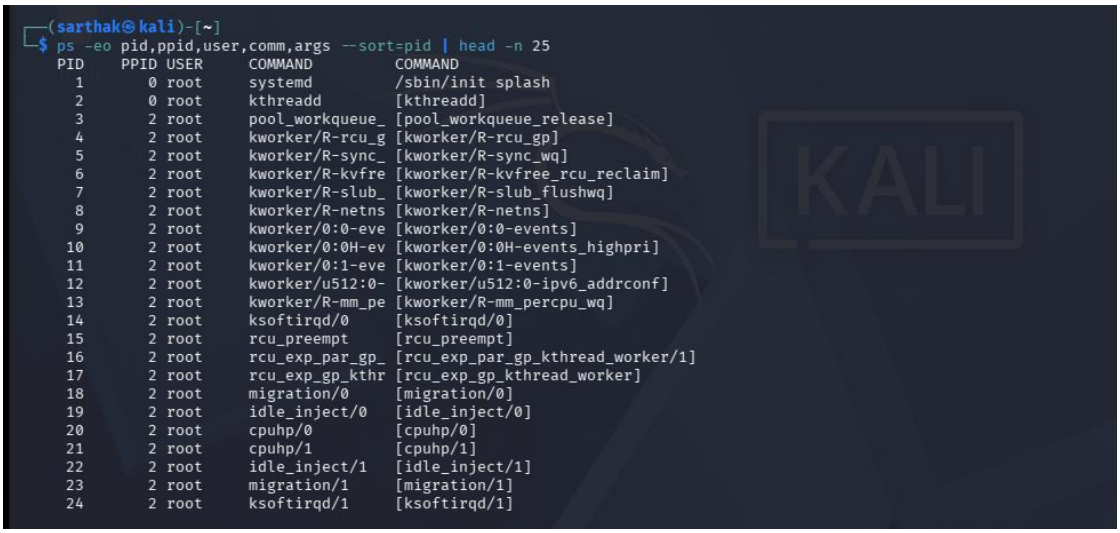
6. Baseline System Observation

Before running the script, the initial system configuration and available utilities were checked.

```
which python3
which ps
which ss
which tcpdump
ip addr
ip route
sudo ss -tulnp
ps -eo pid,ppid,user,comm,args --sort=pid | head -n 25
```

The commands were used to identify available utilities, inspect network interfaces and routes, view listening sockets, and record a sample of running processes. These baseline observations provided context for the later execution.

Figure 4. Baseline tool availability, network configuration, and process information.



7. Python System-Information Test

The selected experiment used a Python script named shell_lab.py. Despite the filename, this version is a system-information test script. It does not implement a shell, create a TCP listener, or communicate with a Netcat client.

7.1 Source Code

```
import platform
import datetime
import os

print("Malware Analysis Sandbox - Lab Test")
print("-----------------------------------")

print("Operating System:", platform.system())
print("Python Version:", platform.python_version())
print("Current User:", os.getenv("USER"))
print("Date and Time:", datetime.datetime.now())

print("Sandbox test completed successfully.")
```

7.2 Execution Procedure

The script was executed from the Kali Linux terminal using the following command:

```
python3 shell_lab.py
```

The script printed the operating system, Python version, current user, and date and time. It then printed the completion message “Sandbox test completed successfully.” The captured execution screenshot showed Linux, Python 3.13.12, user sarthak, and the displayed system date and time.

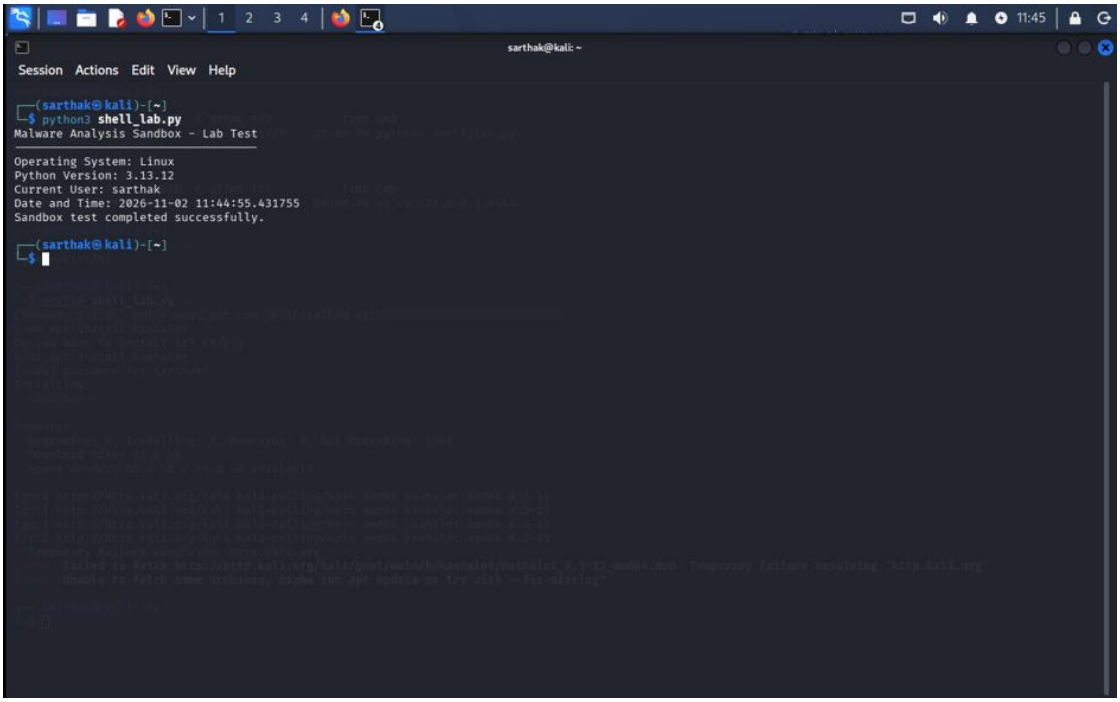
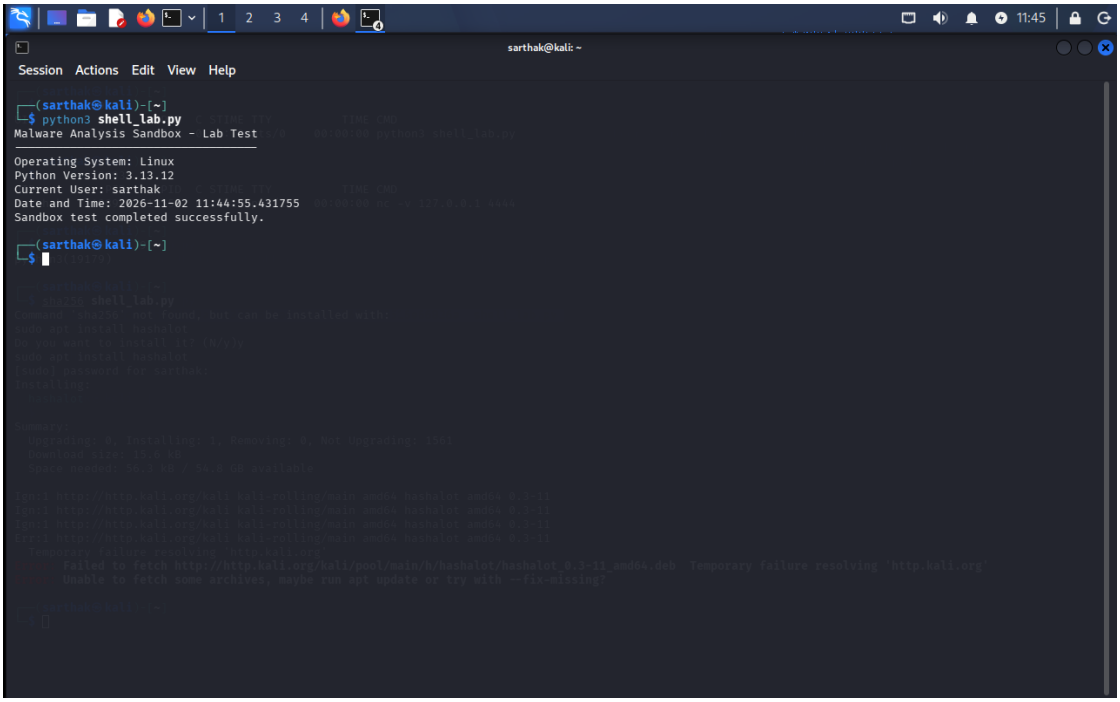


Figure 5. Execution of the Python system-information test script.



7.3 Script Behavior

The platform module provides the operating-system name and Python version. The os module reads the USER environment variable, while the datetime module obtains the current date and time. The script performs local information retrieval and terminates after printing the completion message.

8. Execution Observations

- The script was launched using Python 3 from the Kali Linux terminal.
- The operating system was reported as Linux.
- The Python version displayed was 3.13.12.
- The current user displayed was sarthak.
- The script displayed the system date and time.
- The script printed its completion message and exited.

This version of the script does not import socket, create a TCP server, open port 4444, execute remote commands, or establish a Netcat connection. Therefore, the earlier shell-simulation observations involving Netcat, established TCP sockets, and port 4444 do not apply to this selected script and are intentionally not reported as results of this experiment.

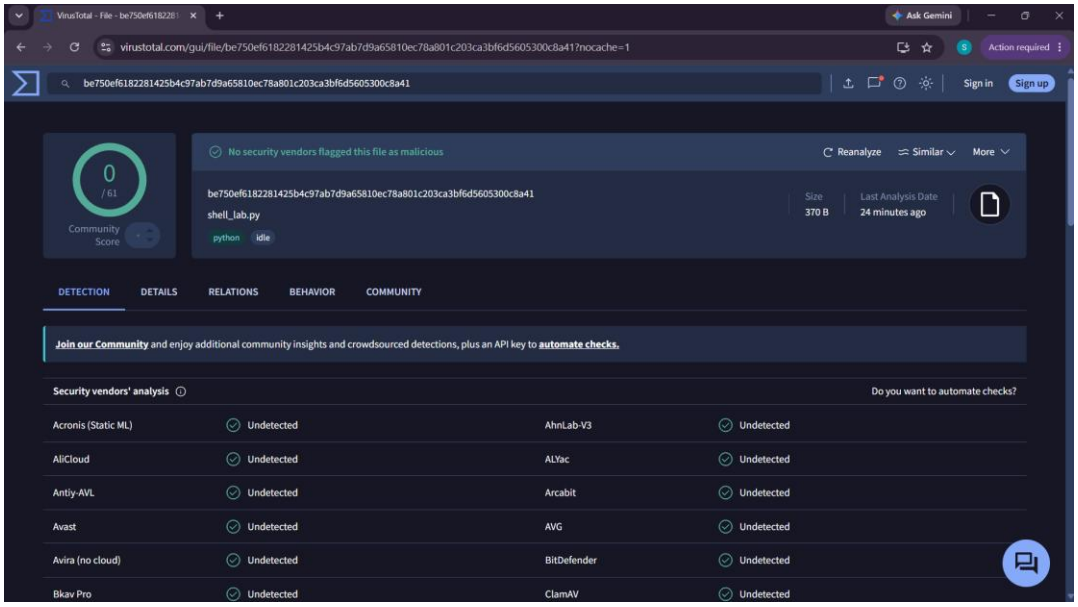
9. SHA-256 and VirusTotal Analysis

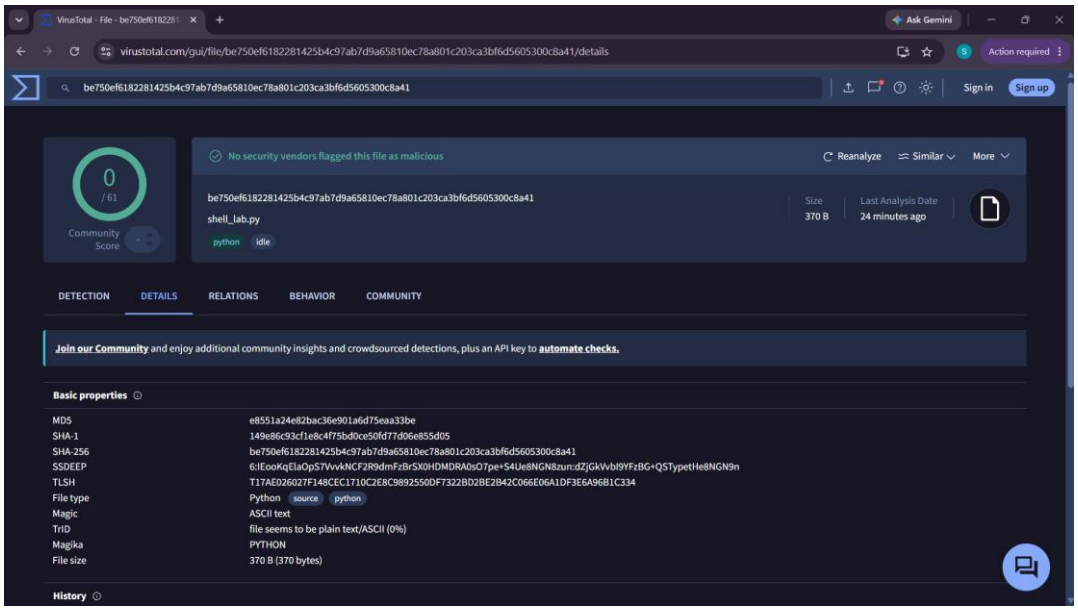
The SHA-256 hash of the system-information test script was generated and searched on VirusTotal. The recorded hash and result were:

```
SHA-256:
be750ef6182281425b4c97ab7d9a65810ec78a801c203ca3bf6d5605300c8a41
VirusTotal result: 0/61
```

The result indicated that 0 of 61 antivirus engines flagged the submitted file at the time of checking. A 0/61 result does not prove that a file is completely safe; it only describes the result returned by the services at the time of the scan. No first-seen date or historical indicators are claimed in this report.

Figure 6. VirusTotal result for the system-information test script.





10. Snapshot Restoration and Verification

After the experiment, the Kali Linux VM was shut down and the “Clean-Kali-Before-Experiment” snapshot was restored using VMware's Snapshot Manager. After restoration, the network route was checked again.

```
ip route
```

The observed route showed the connected 192.168.147.0/24 Host-only subnet and no default route. The experiment script's absence and the absence of experiment-related services should be recorded only where verified in the restored VM.

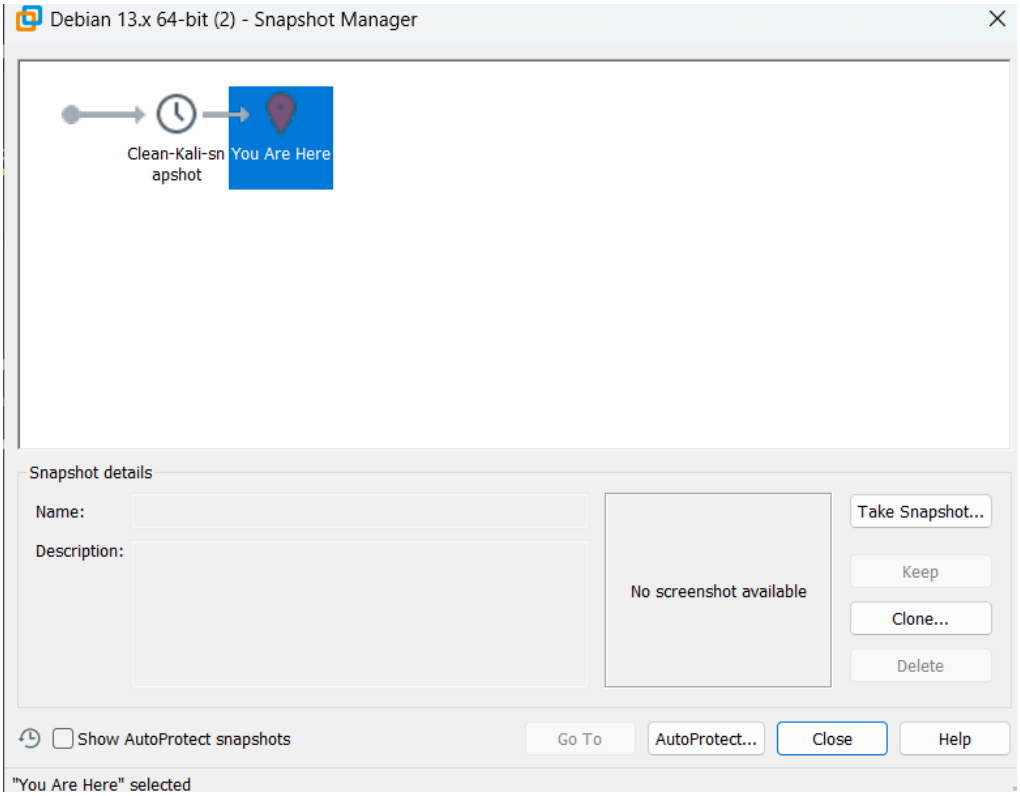


Figure 7. Restored clean snapshot in VMware.

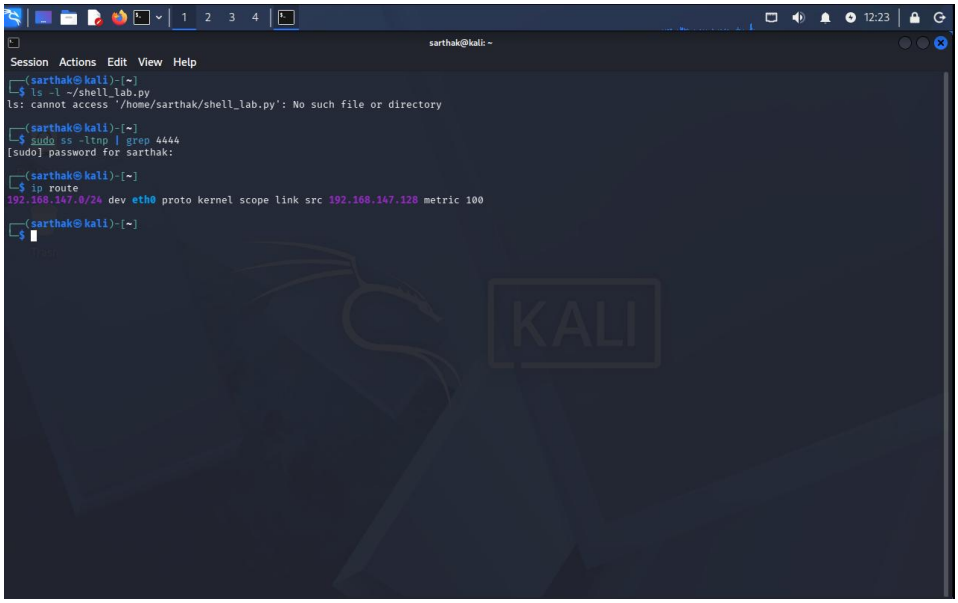


Figure 8. Network route after snapshot restoration.

11. Results and Discussion

The system-information test script executed successfully in the Kali Linux VM and displayed the requested local system details. The recorded VirusTotal lookup returned 0/61 detections for the submitted script hash. The VM was returned to its clean snapshot, and the post-restoration route table showed no default internet route.

The experiment was a basic sandbox-observation exercise, not a full malware-analysis investigation. The script did not exhibit network communication or remote command execution. The practical value was in establishing a controlled VM environment, observing execution, checking a file hash, and restoring a saved VM state.

12. Importance of Sandboxing and Snapshot Recovery

12.1 Sandboxing

Sandboxing provides a controlled environment for executing and observing software. It helps reduce the risk of affecting the host system and makes it easier to monitor program behavior. Network configuration is an important part of the sandbox design, but Host-only mode still permits some communication with the host and other VMs on that virtual network.

12.2 Snapshot-Based Recovery

A snapshot stores the state of a virtual machine at a particular point in time. Restoring a snapshot returns the VM to that saved state, which is useful after experiments that may alter files or configuration. A snapshot is a recovery mechanism, not a substitute for proper isolation or backups.

13. Limitations

- The selected Python script only displays basic system information; it does not simulate a full reverse shell or bind shell.
- Host-only networking prevents a default internet route in this setup but does not guarantee that the guest cannot communicate with the host or other Host-only guests.
- The VirusTotal 0/61 result is time-specific and does not prove that a file is harmless.
- Only the observations and screenshots actually captured during the activity should be treated as experimental evidence.

14. Conclusion

The capstone activity demonstrated how VMware Workstation and Kali Linux can be used to create a controlled environment for a basic software-observation exercise. The system-information test script was executed successfully, its SHA-256 hash was checked on VirusTotal, and the VM was restored to its clean snapshot. The activity highlighted the importance of network configuration, evidence collection, careful interpretation of antivirus results, and snapshot-based recovery.